PROJECT REPORT

SHOPPING CART OPTIMIZER

(KNAPSACK)

DONE BY

NAME:	**REGISTRATION NUMBER:**
T.SAI CHARAN	AP24110011976
T.LEPAKSHI GUNA	AP24110011999
T.HARI VENKAT	AP24110011941
D.MUKUNDA RAJU	AP24110011975
Y.DHARMA TEJA	AP24110011885

CSE 2nd YEAR, FOURTH SEMESTER

SECTION W

SUBMITTED TO

RAKESH RAMA RAJU

CODING SKILLS-2 (CSE 206)

ABSTRACT

The Shopping Cart Optimizer is a console-based application developed in C++ that helps users select the most valuable set of items within a limited budget. The system uses the **0/1 Knapsack Algorithm**, a well-known Dynamic Programming technique, to determine the optimal combination of items that maximizes total value without exceeding the given capacity. The application allows users to input the number of items, their respective weights (costs), and values (importance). Based on this input, the system computes the best possible selection using a dynamic programming approach. Unlike manual selection, which may lead to suboptimal decisions, this system ensures that the best possible value is always achieved.
The project demonstrates the practical application of algorithmic problem-solving techniques in real-world scenarios such as budget planning and resource allocation. It highlights the importance of Dynamic Programming in solving optimization problems efficiently and effectively.

The system is designed with simplicity and efficiency in mind, making it easy for users to interact with and understand the output. By taking dynamic inputs and processing them in real-time, the application ensures flexibility and adaptability for different datasets. The use of structured programming techniques enhances readability and maintainability of the code, allowing future improvements such as graphical user interfaces or database integration.
Furthermore, the project emphasizes the importance of algorithm optimization in modern computing. By comparing different possible combinations and storing intermediate results, the Dynamic Programming approach significantly reduces computational time. This makes the system not only accurate but also efficient for larger inputs. The Shopping Cart Optimizer serves as a strong example of how theoretical concepts can be transformed into practical applications that solve real-life problems effectively.

INTRODUCTION

In today's world, efficient resource allocation plays a critical role in decision-making processes. Whether it is budgeting for shopping, selecting investments, or managing limited resources, finding the optimal solution is essential. The Shopping Cart Optimizer is designed to address such problems using algorithmic techniques.

This system is developed using C++ and focuses on solving the classic **0/1 Knapsack Problem**, where each item can either be selected or not selected. The main objective is to maximize the total value of selected items while ensuring that the total weight does not exceed the given budget.

The project utilizes Dynamic Programming to break down the problem into smaller subproblems and store intermediate results to avoid redundant calculations. This significantly improves efficiency compared to brute-force methods.

The application is simple, user-friendly, and does not require any external database. It takes input from the user and provides the optimal result instantly. This makes it suitable for academic purposes and demonstrations.

In addition to its core functionality, the Shopping Cart Optimizer provides a clear demonstration of how computational problems can be solved efficiently using structured approaches. Instead of checking all possible combinations, which would be time-consuming and inefficient, the system applies a step-by-step optimization technique. This not only reduces execution time but also ensures that the solution is obtained in a systematic and reliable manner.

Moreover, this project highlights the importance of decision-making algorithms in everyday applications. From online shopping platforms to financial planning systems, similar optimization techniques are widely used to maximize benefits under constraints. By implementing this project, users gain practical exposure to such real-world scenarios, enhancing their understanding of how algorithmic solutions can be applied beyond academic problems.

OBJECTIVES

The key objectives of this project are:

- **To develop a system that optimizes item selection within a given budget**
- **To implement the 0/1 Knapsack Algorithm using Dynamic Programming**
- **To maximize the total value of selected items**
- **To minimize computational complexity compared to brute-force methods**
- **To demonstrate real-world application of Dynamic Programming**
- **To provide a simple and user-friendly interface for input and output**
- **To improve problem-solving and algorithm design skills**
- **To ensure efficient memory usage and performance**
- **To validate correctness through multiple test cases**
- **To create a scalable system for future enhancements**
- **To develop a system that provides accurate and optimal results for item selection under budget constraints**
- **To demonstrate the use of tabulation method in Dynamic Programming for efficient computation**
- **To reduce time complexity compared to recursive and brute-force approaches**
- **To design a flexible system that can handle varying numbers of items and capacities**
- **To enhance understanding of optimization problems and their real-world applications**
- **To build a foundation for advanced algorithm-based projects and problem-solving techniques**

MODULES

Module Name	Description
Input Module	Accepts number of items, weights, values, and capacity
Processing Module	Applies Dynamic Programming (Knapsack Algorithm)
DP Table Module	Stores intermediate results for optimization
Decision Module	Chooses maximum value based on inclusion/exclusion
Output Module	Displays maximum achievable value

DATA FLOW DIAGRAM

LEVEL -1 DATA FLOW DIAGRAM

```
+-------------------+
|       USER        |
+-------------------+
          |
          v
+---------------------------+
| Shopping Cart Optimizer   |
|   (Knapsack Algorithm)    |
+---------------------------+
          |
          v
+-------------------+
|      OUTPUT       |
| Maximum Value     |
+-------------------+
```

Explanation

- User provides input (items, weights, values, capacity)
- System processes data using Knapsack Algorithm
- Output displays maximum achievable value

LEVEL -2 DATA FLOW DIAGRAM

```
+------------------+
|      USER        |
+------------------+
          |
          v
+-----------------------+
|   Input Module        |
| (n, weights, values,  |
|   capacity)           |
+-----------------------+
          |
          v
+-----------------------+
|   Processing Module   |
| (DP Table Creation)   |
+-----------------------+
          |
          v
+-----------------------+
| Decision Module       |
| Include / Exclude Item |
+-----------------------+
          |
          v
+-----------------------+
|   Output Module       |
| Maximum Value Display |
+-----------------------+
```

Explanation

- Input is stored in arrays
- DP table is constructed step-by-step
- Each item is evaluated:
  - Include item
  - Exclude item

  Final optimal value is calculated and displayed

CODE & OUTPUTS

```
#include <iostream>
#include <algorithm>
using namespace std;

int knapsack(int n, int capacity, int weights[], int values[]) {
    int dp[100][100];

    for (int i = 0; i <= n; i++) {
        for (int w = 0; w <= capacity; w++) {
            if (i == 0 || w == 0)
                dp[i][w] = 0;
            else if (weights[i - 1] <= w)
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                );
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    return dp[n][capacity];
}

int main() {
    int n, capacity;

    cout << "Enter number of items: ";
    cin >> n;

    int weights[n], values[n];

    cout << "Enter weights of items:\n";
    for (int i = 0; i < n; i++) cin >> weights[i];

    cout << "Enter values of items:\n";
    for (int i = 0; i < n; i++) cin >> values[i];

    cout << "Enter maximum budget (capacity): ";
    cin >> capacity;

    int result = knapsack(n, capacity, weights, values);

    cout << "\nMaximum value you can get = " << result << endl;

    return 0;
}
```

OUTPUTS:

Sample Input:

```
Enter number of items: 5
Weights: 7 8 2 9 12
Values: 2 9 13 4 3
Capacity: 12
```

Output:

```
Maximum value you can get = 22
```

ALGORITHM USED

- Dynamic Programming
- 0/1 Knapsack Algorithm
- Time Complexity: **O(n × capacity)**
- Space Complexity: **O(n × capacity)**

CONCLUSION

The Shopping Cart Optimizer is an effective implementation of the 0/1 Knapsack Algorithm using Dynamic Programming. The project successfully demonstrates how algorithmic techniques can be applied to solve real-world optimization problems in a structured and efficient manner

The system ensures that users obtain the maximum possible value within a given budget, eliminating inefficient and time-consuming manual selection methods. It clearly illustrates how Dynamic Programming handles complex problems involving overlapping subproblems and optimal substructure by storing intermediate results and avoiding redundant calculations. This not only improves efficiency but also guarantees accuracy in the final output.

In addition, the project strengthens the understanding of fundamental programming concepts such as arrays, loops, and conditional statements in C++. It also enhances logical thinking and problem-solving abilities by encouraging a systematic approach to tackling optimization problems. The modular design of the program makes it easy to understand, debug, and extend, which is essential for developing scalable software applications.

Furthermore, this project provides a strong foundation for future advancements and enhancements. It can be extended by incorporating features such as displaying the selected items, implementing a graphical user interface, or integrating database support for handling larger datasets.

The project also emphasizes the importance of computational efficiency and resource management in modern software development. By reducing time complexity compared to brute-force approaches, the solution demonstrates how optimized algorithms can significantly improve performance, especially when dealing with large inputs and constrained environments.